

#UNIT-2

#Why Traffic Modeling is Important:

- (i) Capacity planning
- (ii) Cost optimization
- (iii) Bottleneck identification
- (iv) Foundation of architecture decisions

#Key concepts in Traffic Modeling:

- (i) DAU (Daily Active Users): Number of users active per day
- (ii) MAU (Monthly Active Users): Number of users active per month
- (iii) Peak concurrent users: Maximum users active at the same time
- (iv) Read vs Write Ratio: Most large systems are read heavy.
Typical ratios include 90:10 or 99:1

Q. Step by step traffic estimation:

Ans Step 1: Estimate total users

Eg - 100 million total users, 10% DAU = 10 million daily active users

Step 2: Estimate Requests Per User

Assume each active user performs 50 actions per day, each action triggers 2 backend requests.

Total requests per day = 1 billion

Step 3: Requests Per Second (RPS)

Total seconds per day = 86,400

Average RPS ≈ 11574

Step 4: Peak Traffic Estimation

Peak traffic is usually 4-5 times the average

Peak RPS ≈ 58000

#Data Storage Estimation and Bandwidth:

(a) Data Storage Estimation:

Assumed each write stores 1 KB of data

Daily data ≈ 500 GB

Monthly data ≈ 15 TB

Yearly data ≈ 180 TB

(b) Bandwidth Estimation:

Assume avg response size is 10 KB

Peak bandwidth ≈ 580 MB/sec (≈ 4.6 Gbps)

(c) Cache Impact on Traffic:

With an 80% cache hit ratio, effective database reads drop significantly

(d) Traffic Spikes & Burst Modeling:
Systems must handle flash sales, viral events, and sudden spikes using auto scaling, caching & rate limiting

Regional Traffic Distribution, Component Level Estimation and Interview Perspective:

(a) Regional Traffic distribution:

Traffic may be split across regions. This affects deployment and replication strategies

(b) Component Level Estimation:

- API Servers: If one server handles 2000 RPS, ~40 servers are needed for peak server traffic

- Databases: Must handle peak writes and scaled reads via replicas

- Caches: Must store hot data efficiently

(c) Interview Perspective:

Explains assumptions clearly & focus on reasoning rather than exact numbers

Common mistakes:

- (i) Designing for average traffic
- (ii) Ignoring cache impact
- (iii) Underestimating bandwidth
- (iv) No buffer for growth

Real World Examples:

- (i) Social media platforms
- (ii) E-commerce platforms

Key Challenges in Scaling Systems:

(i) Scalability:

You can add more servers when traffic grows.

Load balancer automatically starts sending traffic to new server

(ii) Fault Tolerance:

Even if a server goes offline, users never notice

Load balancer re-routes traffic immediately

(iii) High Throughput:

Overall system performance increases because workload is evenly shared

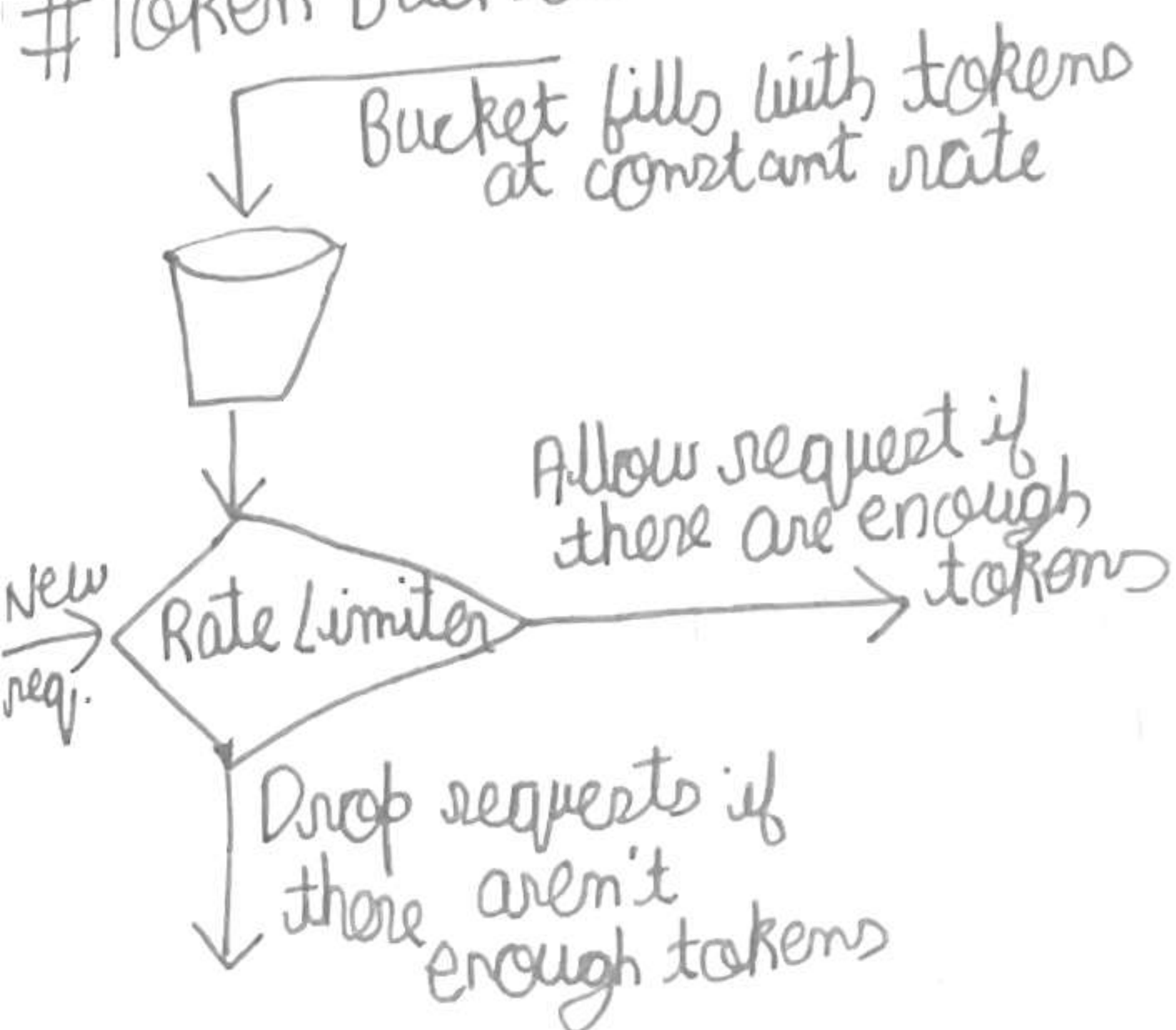
#Rate Limiting -

Helps protect services from being overwhelmed by too many requests from a single user or client.
Limits the quantity or frequency of client requests to prevent overload, maintain stability, and ensure fair resource distribution.

- (i) Reduce possibility of resource abuse and denial-of-service (DoS) attack
- (ii) Frequently used in web servers, APIs, network traffic management, and database access
- (iii) eg- AWS API Gateway Throttling, AWS WAF (Web Application Firewall), Azure API Management Rate Limiting

A rate limiter is a component that controls the rate of traffic or requests to a system.

#Token Bucket -



(i) Allocates tokens at a fixed rate into a 'bucket'

(ii) Each request consumes a token from the bucket, and requests are only allowed if there are sufficient tokens available

(iii) Unused tokens are stored in the bucket, up to a maximum capacity

(iv) Provides a simple & flexible way to control the rate of requests and smooth out bursts of traffic

How it works:

- 1) Imagine a bucket that holds tokens
- 2) Bucket has a max capacity of tokens
- 3) Tokens are added to the bucket at a fixed rate (eg- 10 tokens/s)
- 4) When a request arrives, it must obtain a token from the bucket to proceed
- 5) If there aren't tokens, request is dropped

Pros

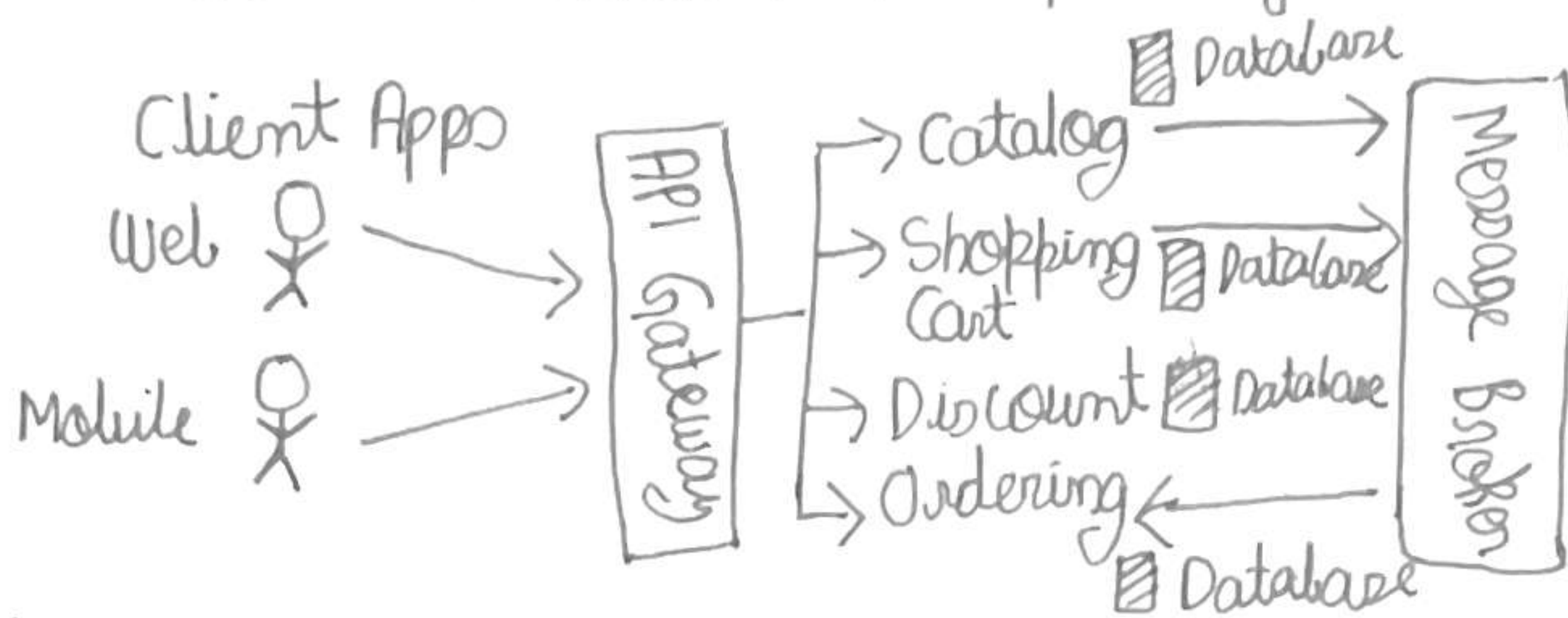
- Simple to implement & understand
- Allows bursts of requests up to the bucket's capacity

Cons

- Memory usage scales with the no. of users if implemented per user
- Does n't guarantee a perfectly smooth rate of requests

API Gateway -

- (i) A reverse proxy that sits at the edge of your system and handles all client requests before they reach back end services.
- (ii) Centralizes cross cutting concerns (routing, auth, rate limiting, transformation, monitoring)
so microservices can remain simple & focused



(iii) Why:

- Simplifies client to service interaction
- Reduces network complexity
- Provides unified security
- Enables monitoring & traffic control
- Supports microservices scalability

(iv) Core Functions:

- Request routing & load balancing
- Authentication & authorization : validate tokens (JWT/OAuth2), API keys, mTLS
- Rate limiting & throttling
- Caching
- Request/response transformation: JSON $\leftrightarrow$ XML, field renamed, versioning adapters
- Protocol translation: HTTP $\leftrightarrow$ gRPC $\leftrightarrow$ WebSocket
- Aggregation/composition: combine responses from multiple services into one response
- Observability: logging, metrics, traces, request IDs
- Security: WAF rules, input validation, IP filtering
- Traffic shaping/canary & blue green routing: route a % of traffic to new versions

(v) Architecture:

- (a) Central entry point
- (b) Request handling & routing
- (c) Integration with service discovery: gateway uses a service registry to dynamically locate microservices
- (d) High availability setup
- (e) Security layer (TLS termination, OAuth2/JWT checks, WAF rules & IP filtering)

(f) Centralized Cross Cutting Features:

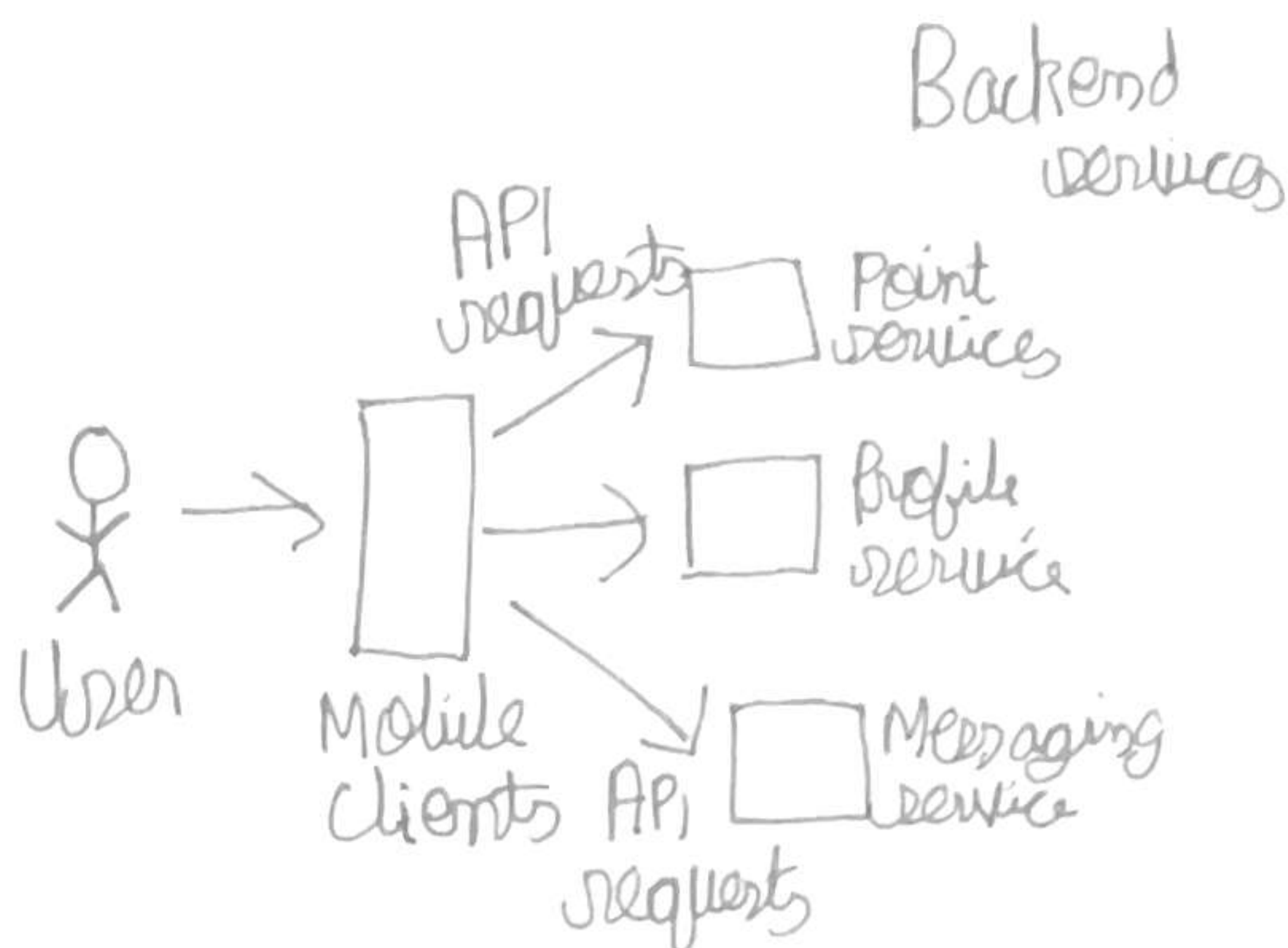
- Authentication & authorization
- Rate limiting & throttling
- Load balancing
- Logging & monitoring
- Request/response transformation
- Caching

(g) Protocol transformation:

- Allow communication between different technologies
- HTTP $\leftrightarrow$ gRPC
- HTTP $\leftrightarrow$ WebSockets

(vi) Request Flow:

- client sends request
- Gateway performs auth
- Applies rate limits
- Routes request
- Aggregates response
- Sends result



(vii) Key Components:

- Routing engine:** directs incoming requests to the correct backend microservices based on URL, method, or rules
- Security layer (Auth & Auth2):** handles authorization & authentication
- Policy engine:** applies rules such as rate limiting, throttling, quotas, and access control policies
- Caching layer:** stores frequently accessed responses
- Load Balancer:** distributes traffic evenly across multiple service instances to ensure reliability & scalability

(viii) Benefits:

- Centralized governance
- Better security
- Higher performance
- Simplifies services
- Early versioning

(ix) Challenges:

- Possible single point of failure
- Added latency
- Complex setup
- Requires scaling strategy

#Content Delivery Network (CDN) -

Def (i) A distributed network of servers that deliver web content based on the user's geographic location

(ii) Reduces latency by serving data from the nearest server

(iii) Improves website performance, availability & scalability

(iv) Commonly delivers images, videos, CSS, JS, and web pages

(v) Components:

- Edge Servers = Cache content near users (PoP locations)

- Origin Server = Stores the original content

- DNS = Redirects users to the nearest edge server

(vi) Steps:

- User requests website content

- DNS resolves request to nearest CDN edge server

- Edge server checks cache

- If cached → content served instantly

- If not cached → fetched from origin server & stored

- Future requests served from cache

(vii) Benefits:

- Faster load time due to reduced latency

- Reduced origin server load through caching

- Higher availability & reliability

- Better scalability

- Improved security

(viii) Challenges:

- Integration & cache config complexity

- Potential higher cost for large traffic

(ix) Use Cases:

- Web acceleration

- Video streaming/OTT platforms

- Online gaming

- News & media distributions

- Software updates

(x) Popular CDN Providers:

- Akamai

- Cloudflare

- Fastly

- Amazon CloudFront

- Google Cloud CDN

- Microsoft Azure CDN

Global Caching -

(i) A centralized caching system shared across multiple servers or applications to store frequently accessed data

(ii) Key characteristics:

- Shared cache
- Scalable infrastructure
- Reduces database load
- Improves response speed
- Requires network communication & complex setup

(iii) Steps:

- Application requests data
- System checks global cache
- Cache hit → data returned immediately
- Cache miss → data fetched from database
- Data stored in cache with
- Load balancer distributes requests across nodes

(iv) Use Cases:

- Microservices Architectures
- CDN
- Large scale distributed applications
- Google Global Cache

Comparison -

Local Cache	Global Cache
(i) Stored inside single application	(i) Shared across multiple systems
(ii) Very fast access	(ii) Slight network latency
(iii) Limited scope	(iii) Scalable & distributed
(iv) Simple setup	(iv) More complex infrastructure

Message Queues -

- Communication mechanism that enables different parts of a system to send messages asynchronously
- Acts as an intermediary that temporarily holds messages sent from producers (or publishers) & delivers them to consumers (or subscribers)



- Components:

(a) Producer/Publisher

- Sends messages to the queue
- Pushes messages directly into the queue

(b) Consumer/Subscriber

- Reads messages from the queue
- Pull messages from queue & process them

(c) Queue

- Stores messages ~~that~~ until they are consumed

(d) Broker/Queue Manager

- Manages the message queue
- Handles delivery of message
- Ensures that messages are routed correctly between producers & consumers

(e) Message

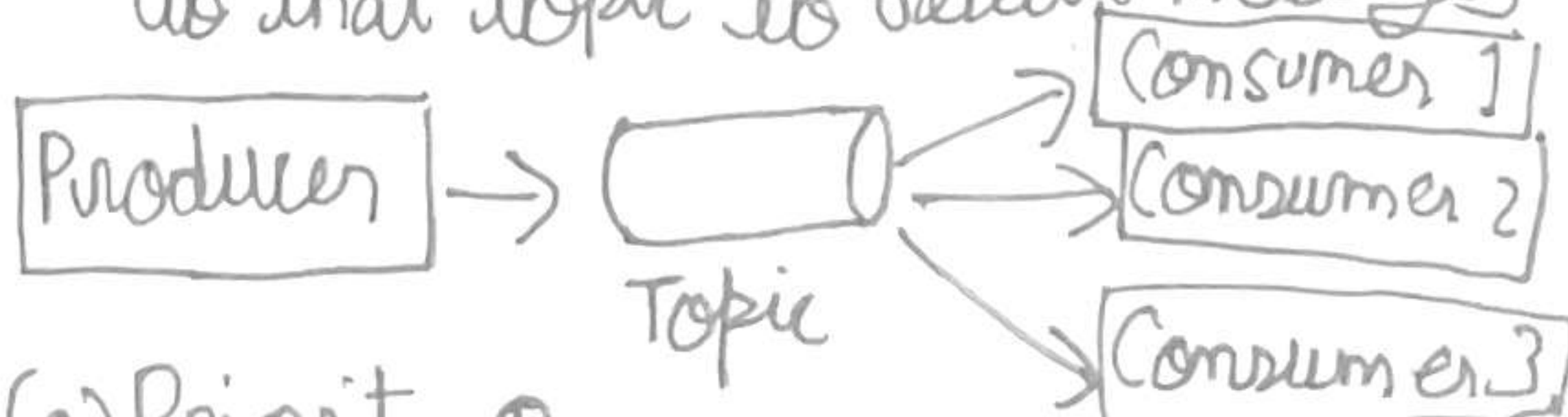
- Unit of data sent through the queue
- Typically contain the payload (actual data) & metadata (headers, timestamps, priority)

- Types:

(a) Point-to-Point (P2P) Queue
Messages are sent from one producer to one consumer

(b) Publish/Subscribe Queue

Messages are published to a topic, & multiple consumers can subscribe to that topic to receive messages



(c) Priority Queue

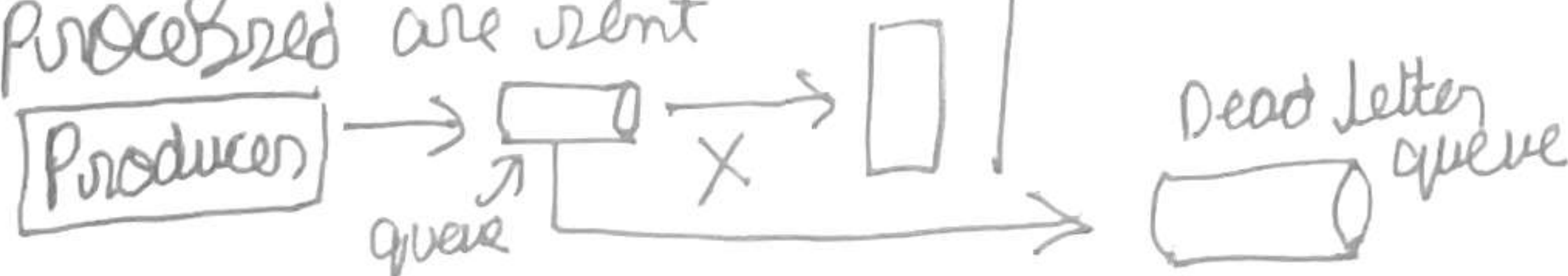
Messages in the queue are assigned priorities, & higher priority messages are processed before lower quality ones

- Workflow:

- Message creation
- Message enqueue
- Message storage
- Message dequeue
- Acknowledgement
- Message deletion

(d) Dead Letter Queue

Messages that cannot be consumed are sent



- Advantages:

- Decoupling
- Asynchronous processing
- Load balancing
- Fault tolerance
- Scalability
- Throttling

- Applications:

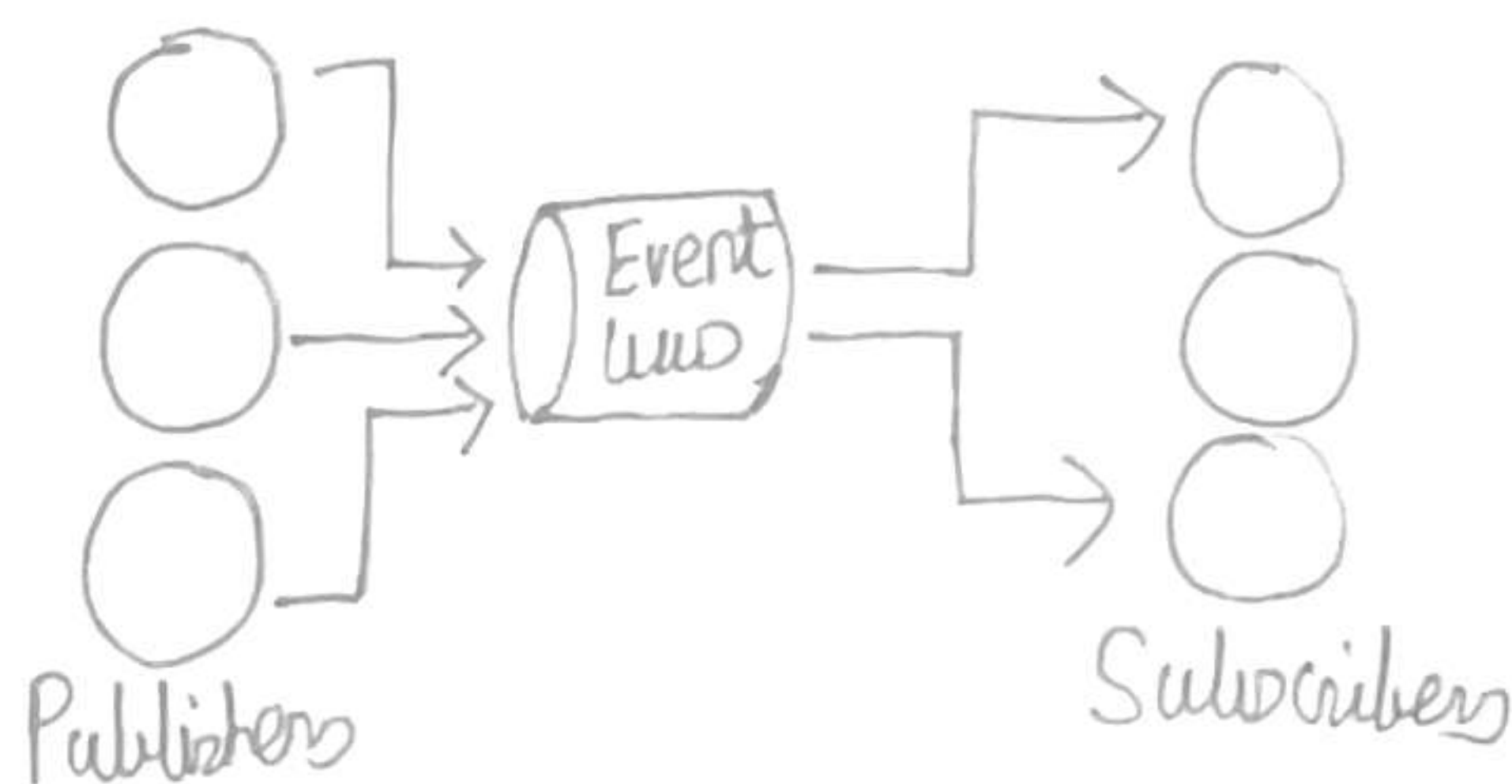
- Microservices architecture
- Task scheduling & background processing
- * - Event driven architecture
- Reliable communication

- Popular Message Queue Systems:

- RabbitMQ
- Amazon SQS
- Google Cloud Pub/Sub
- ActiveMQ
- Redis Streams

Event Driven System -

- An architecture where components communicate by producing and reacting to events
- Services do not call each other directly instead they communicate indirectly via events.



Q. What is an event?

Ans (i) Represents a change in system state

(ii) Contain event name, timestamp, payload

(iii) Producers generate events

(iv) Events are sent to a message broker

(v) Consumers react to events independently

(vi) Makes the system loosely coupled & highly scalable

Core Components of EDS -

(a) Event Producers

- Detect something happening & publish an event
- Generates events on state change
- Producers are decoupled from consumers
- eg - Order, User, Payment services

(b) Event Broker/Message Queue

- Heart of the system. Store & deliver events
- Central event distribution system
- Ensures durability, scalability, ordering & persistence
- eg - Kafka, RabbitMQ, AWS SNS

- (c) Event Consumers
- Services that listen for the events & take appropriate actions
 - Subscribes to specific events
 - Processes them independently

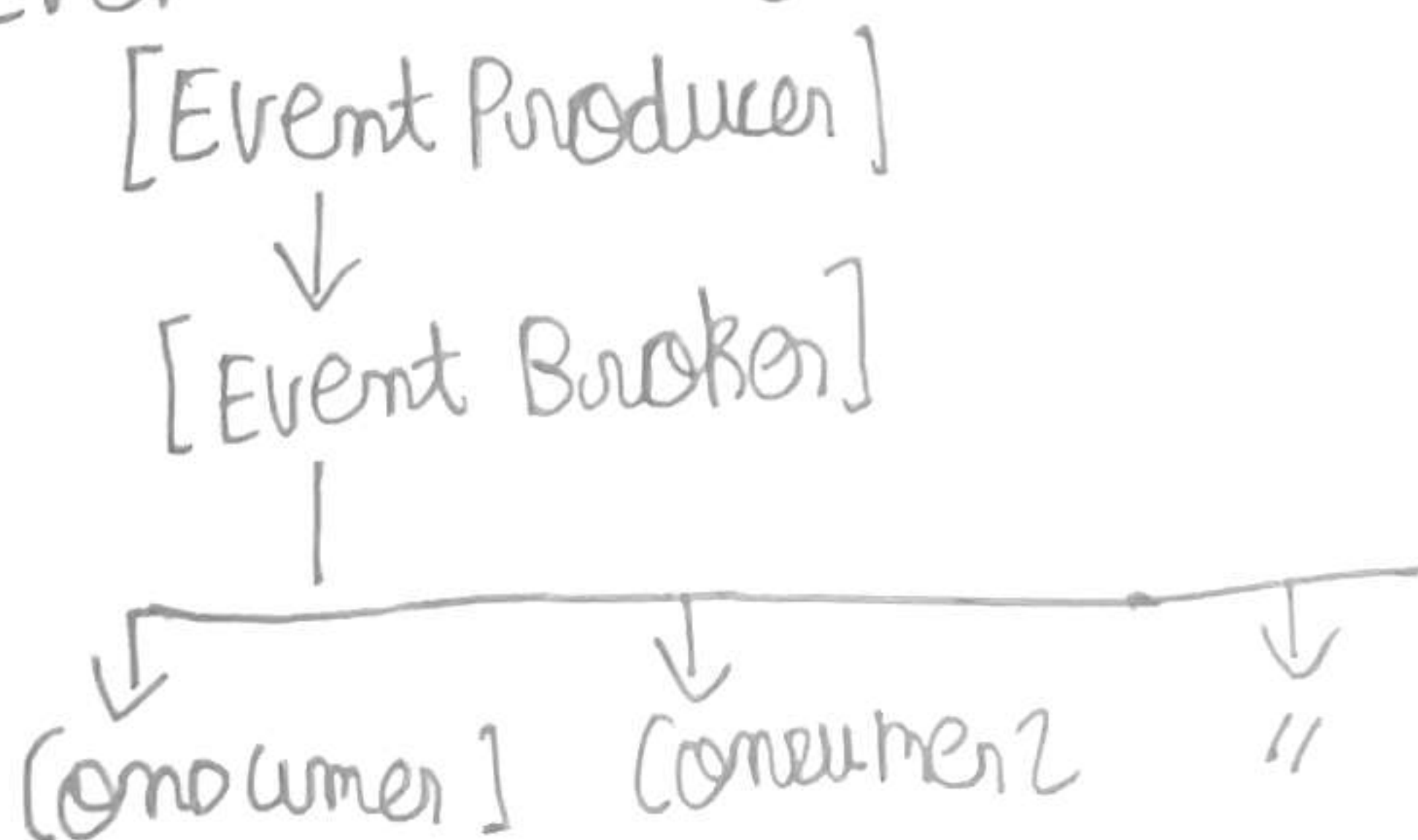
Event producers



Event Driven System Working -

- 1) An event occurs:
 - A service detects a change in state & generates an event
 - Components
 - Event name
 - Time stamp
 - Payload (data)
- 2) Event producer publishes the event:
 - Producer sends the event to the Event Broker
 - Producers do not know who will consume the event $\Rightarrow$ they are fully decoupled
- 3) Event Broker receives & stores the event:
- 4) Event Consumers Subscribe to Events
 - Consumers listen for events they are interested in
 - Consumers work independently & in parallel
- 5) Consumers Process Events asynchronously:
 - After receiving the event, each consumer performs its own task
 - Asynchronous: producer does not wait for the consumer
- 6) Event Flow continues for other services:
 - Multiple consumers can respond to the same event
 - No consumer is aware of the others
- 7) System achieves loose coupling & scalability:

Event Driven System Flow Diagram -



Types of EDA -

1) Event Notification

- Event only signals something happened
- Consumer fetches data separately

2) Event Carried State Transfer

- Event includes full data needed by consumers

3) Event Sourcing

- System stores a log of all events as the source of truth
- Application state = replayed events

Benefits of EDS -

- High scalability
- Loose coupling
- Fault tolerance
- Easy to extend with new services

Challenges of EDS -

- Complex debugging
- Event duplication possible
- Difficult ordering
- Eventual consistency
- Complex error handling

OAuth (Open ~~Auth~~ Authorization) -

- Open standard that allows a user to grant a 3rd application limited access to their resources without sharing their password

- eg - Login with Google

- Why

- Prevents password sharing
- Provides limited, controlled access
- Supports secure login for mobile, web, APIs
- Permissions can be revoked anytime

- Important terms -

(a) Resource Owner -

- User who owns the data
- eg - You

(b) Client application

- App requesting access
- eg - Spotify, Zoom.

(c) Authorization server

- Generates tokens after validating the user

- eg - Google / Facebook login service

(d) Resource Server

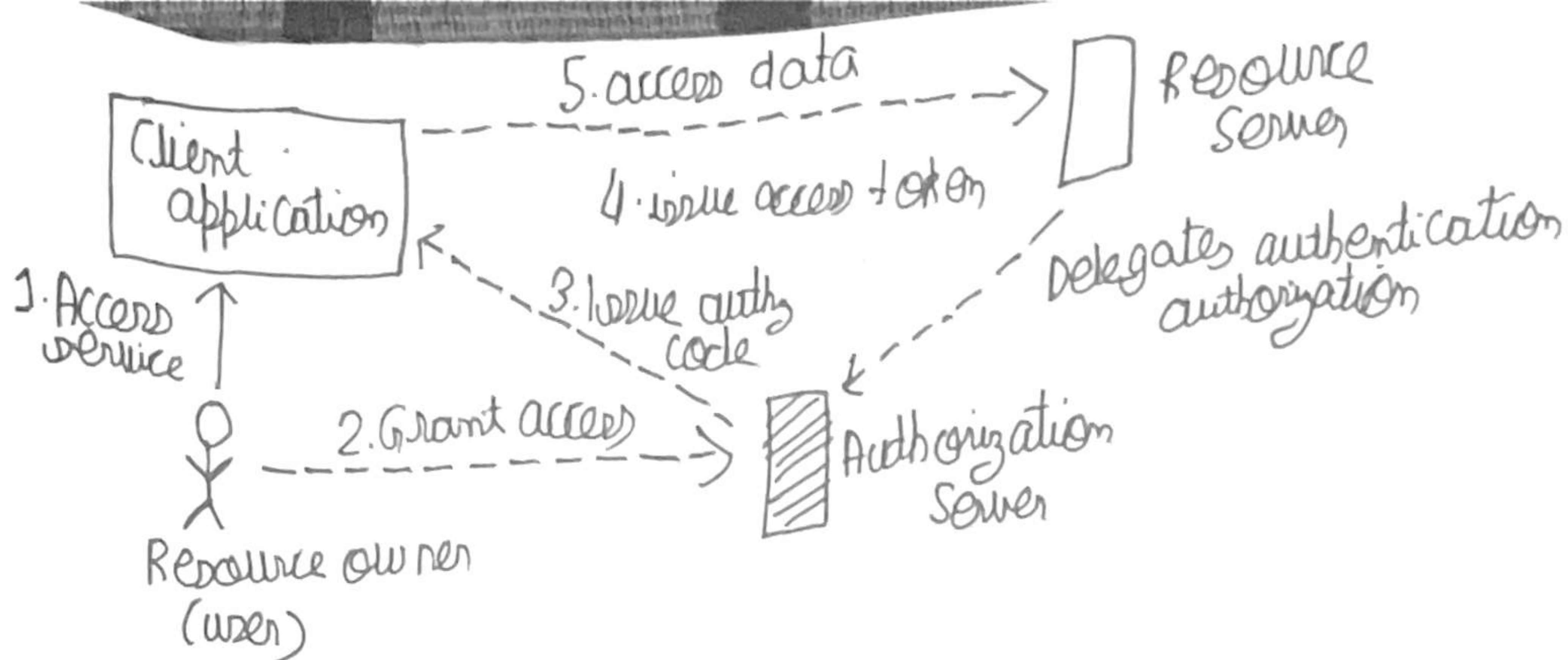
- Server containing user data
- eg - Google APIs (profile, email)

(e) Access Token

- Temporary token allowing limited access
- eg - Token that lets an app read your Gmail profile

(f) Refresh Token

- Used to get a new access token when old one expires



Access Token

- (i) A short lived token
- (ii) Allows client apps to access specific user data from the resource server
- (iii) Usually valid for minutes to hrs
- (iv) Sent in API requests
- (v) Represents the permissions granted by the user
- (vi) Can be revoked anytime
- (vii) Eg - A token allowing an app to read your Google Profile & email

OAuth Flow -

- 1) User requests login
- 2) User authenticates
- 3) User Grants permission
- 4) Authorization code sent to client
- 5) Client Exchanges Code for Access Token
- 6) Client uses the access token
- 7) Refresh token for long sessions
- 8) Logout on revocation

OAuth Grant Types -

- (i) Authorization code grant - Recommended
- (ii) Client Credentials grant - Server to Server
- (iii) Device Code flow - TV/IoT
- (iv) Password grant - Deprecated
- (v) Implicit grant - Deprecated

Access Token Usage -

- (i) Used for authenticated APIs
- (ii) eg - /user/info returns email & profile
- (iii) Token expires → refreshed

Benefits -

- (i) Secure token based access
- (ii) No password sharing
- (iii) Fine grained permissions
- (iv) Tokens revocable anytime
- (v) Supported by major providers